

UNIVERSIDAD NACIONAL DE CAÑETE
FACULTAD DE INGENIERÍA
ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMAS



**VII Jornada de Exposición de Productos Académicos de la Escuela
Profesional de Ingeniería de Sistemas**

PROYECTO

**GESTOR DINÁMICO DE HORARIOS, ESPACIOS, DOCENTES Y CURSOS
PARA LA FACULTAD DE INGENIERÍA DE SISTEMAS.**

AUTOR(ES):

SERGIO ROBERTO FERNÁNDEZ ANGULO

DOCENTE:

DAVID CENTENO CÁCERES

DISCIPLINA

PROGRAMACIÓN Y DESARROLLO DE SOFTWARE

CURSO

ALGORITMOS Y FUNDAMENTOS DE PROGRAMACIÓN

San Vicente de Cañete, Perú, 2026

Parte 1: Documentación de presentación

Documentación detallada del proyecto Schedule

Portada

Proyecto: Schedule Desktop App Tipo de sistema: Aplicación académica full-stack para gestión de horarios Área: Escuela Profesional de Ingeniería de Sistemas Versión del documento: Presentación técnica y funcional Fecha: Julio de 2026

Resumen para presentación

Schedule es un sistema académico desarrollado para apoyar la organización de horarios, cursos, docentes, ambientes, usuarios y reglas institucionales. El proyecto combina un backend Java Spring Boot, un frontend React con Vite y un contenedor de escritorio Electron. La persistencia se realiza con PostgreSQL, levantado localmente mediante Docker Compose.

El sistema está orientado a resolver un problema real de gestión académica: ordenar información dispersa, evitar conflictos de horarios, controlar asignaciones docentes y centralizar reglas de planificación. A diferencia de una maqueta estática, Schedule cuenta con autenticación, roles, API REST, datos persistentes, validaciones de negocio y una arquitectura preparada para evolucionar.

Justificación del proyecto

La elaboración manual de horarios académicos suele requerir revisar docentes, disponibilidad, ciclos, ambientes, cursos, turnos y reglas institucionales. Este proceso puede generar errores como cruces de docentes, uso simultáneo de ambientes, cursos ubicados fuera de su turno permitido o falta de sincronización entre asignaciones.

Schedule busca reducir esos riesgos mediante una plataforma unificada donde la información se registra, consulta y valida desde una misma aplicación. El proyecto también permite demostrar conocimientos de desarrollo full-stack, arquitectura por capas, persistencia relacional, autenticación segura, consumo de API desde frontend y empaquetado de una experiencia de escritorio.

Objetivo general

Desarrollar una aplicación académica capaz de gestionar información institucional relacionada con docentes, cursos, ambientes, usuarios y horarios, aplicando reglas de negocio para apoyar la generación y revisión de propuestas de horario.

Objetivos específicos

- Implementar una API REST con Spring Boot para administrar entidades académicas.
- Persistir información real en PostgreSQL.
- Proteger rutas mediante autenticación JWT y roles de usuario.

- Construir una interfaz React para operar el sistema de forma visual.
- Permitir la gestión de docentes, cursos, ambientes, usuarios y reglas horarias.
- Representar restricciones académicas como ciclos, turnos, bloques lectivos y no lectivos.
- Integrar la aplicación web dentro de un contenedor Electron para uso de escritorio.
- Documentar la arquitectura, ejecución, reglas y funcionamiento del sistema.

Alcance

El alcance actual incluye administración de usuarios, autenticación, gestión de docentes, cursos, ambientes, jefes de práctica, reglas horarias y visualización/generación de propuestas de horario. También incluye documentación técnica para instalación, ejecución y mantenimiento.

El sistema está pensado principalmente para desarrollo y demostración funcional. Para un despliegue productivo se recomiendan ajustes adicionales como secretos seguros, migraciones de base de datos, backups, configuración de dominio, políticas de CORS y empaquetado final de Electron.

Audiencia del documento

Este documento está dirigido a docentes evaluadores, jurados, compañeros de proyecto, usuarios técnicos y personas que necesiten entender el funcionamiento general de Schedule sin revisar directamente todo el código fuente.

ÍNDICE GENERAL

Parte 1: Documentación de presentación.....	2
Portada.....	2
Resumen para presentación.....	2
Justificación del proyecto.....	2
Objetivo general.....	2
Objetivos específicos.....	2
Alcance.....	3
Audiencia del documento.....	3
1. Resumen general.....	6
2. Objetivo funcional del sistema.....	7
3. Estructura del repositorio.....	8
4. Requisitos de entorno.....	11
5. Base de datos.....	12
6. Configuración del backend.....	13
7. Dependencias principales.....	14
8. Ejecución en desarrollo.....	16
9. Autenticación y usuarios.....	17
10. API REST.....	18
11. Modelo de dominio.....	21
12. Reglas académicas de horarios.....	23
13. Reglas por ciclo.....	25
14. Reglas de docentes.....	27
15. Horarios y generación de propuestas.....	28
16. Frontend: pantallas principales.....	29
17. Cliente HTTP del frontend.....	31
18. Sesión frontend.....	32
19. Sincronización curso-docente.....	33
20. Plan de estudios y carga horaria.....	34
21. Documentación existente recomendada.....	35
22. Flujo recomendado para desarrollo.....	36
23. Reglas para agentes en este repositorio.....	37
24. Problemas frecuentes.....	38
25. Consideraciones para producción.....	39
26. Resumen ejecutivo final.....	40
Parte 2: Explicación del funcionamiento.....	41
Portada.....	41
1. Explicación general.....	42
2. Partes principales del sistema.....	43
3. Cómo funciona cuando se abre la aplicación.....	44
4. Cómo funciona el inicio de sesión.....	45
5. Cómo funciona la seguridad.....	45
6. Como viajan los datos.....	46
7. Como se guardan los datos.....	47

8. Cómo funciona la gestión de usuarios.....	48
9. Cómo funciona la gestión de docentes.....	49
10. Cómo funciona la gestión de cursos.....	50
11. Cómo funciona la gestión de ambientes.....	51
12. Como funcionan las reglas horarias.....	52
13. Cómo funciona la asignación docente-curso.....	53
14. Cómo funciona la generación de horarios.....	54
15. Cómo funciona Docker en este proyecto.....	55
16. Cómo funciona Electron.....	56
17. Ejemplo completo explicado.....	57
18. Explicación final para exposición.....	58
Parte 3: Funcionamiento interno.....	59
Portada.....	59
1. Idea general del funcionamiento.....	60
2. Como se inicia el sistema.....	60
3. Como se comunican las capas.....	63
4. Funcionamiento del login.....	64
5. Funcionamiento de la autorización.....	66
6. Funcionamiento del cliente HTTP.....	67
7. Funcionamiento de la persistencia.....	67
8. Funcionamiento de los CRUD.....	69
9. Funcionamiento de usuarios.....	71
10. Funcionamiento de docentes.....	72
11. Funcionamiento de cursos.....	72
12. Funcionamiento de ambientes.....	74
13. Funcionamiento de jefes de práctica.....	75
14. Funcionamiento de reglas horarias.....	76
15. Funcionamiento de turnos por ciclo.....	77
16. Funcionamiento de asignaciones curso-docente.....	78
17. Funcionamiento de horarios.....	79
18. Funcionamiento visual del frontend.....	80
19. Funcionamiento del backend por capas.....	81
20. Funcionamiento de Docker en el proyecto.....	82
21. Funcionamiento de Electron.....	83
22. Qué pasa cuando ocurre un error.....	84
23. Ejemplo completo: crear un usuario.....	85
24. Ejemplo completo: asignar docente a curso.....	86
25. Ejemplo completo: modificar horarios del día.....	87
26. Ejemplo completo: generar horario.....	88
27. Fuente de verdad del sistema.....	89
28. Resumen final del funcionamiento.....	90

1. Resumen general

Schedule es una aplicación para gestionar programación académica de la Escuela Profesional de Ingeniería de Sistemas. El sistema integra una API backend, una interfaz web y un contenedor de escritorio.

La arquitectura principal está dividida en tres partes:

Carpeta	Tecnología	Función
backend/	Java 21, Spring Boot, Maven	API REST, seguridad, reglas de negocio y persistencia
frontend/	React, Vite, Tailwind CSS	Interfaz de usuario para administrar docentes, cursos, ambientes, usuarios, reglas y horarios
desktop/	Electron	Envoltorio de escritorio que carga la interfaz frontend

La base de datos usada en desarrollo es PostgreSQL 16 levantada con Docker Compose.

2. Objetivo funcional del sistema

El objetivo de Schedule es facilitar la organización académica mediante:

- Gestión de usuarios con login real y roles.
- Gestión de docentes.
- Gestión de cursos del plan de estudios.
- Gestión de ambientes o espacios.
- Gestión de jefes de práctica.
- Asignación de docentes a cursos por turno.
- Asignación de cursos a ambientes.
- Configuración de franjas horarias del día académico.
- Generación de propuestas de horario por ciclo.
- Validación de reglas académicas para evitar asignaciones incompatibles.
- Persistencia de datos en PostgreSQL.

El sistema no es solo una maqueta visual. El backend guarda datos reales, aplica validaciones y expone endpoints REST protegidos por JWT.

3. Estructura del repositorio

Schedule/

```
|— backend/
|  |— pom.xml
|  └─ src/main/java/com/example/schedule/
|— frontend/
|  |— package.json
|  └─ src/
|— desktop/
|  |— package.json
|  └─ main.js
|— docs/
|— docker-compose.yml
|— README.md
└─ AGENTS.md
```

3.1. backend/

Contiene la aplicación Spring Boot. Sus responsabilidades principales son:

- Exponer la API REST.
- Gestionar autenticación y autorización.
- Validar reglas de negocio.
- Persistir entidades con JPA.
- Conectarse a PostgreSQL.
- Inicializar datos base.
- Calcular propuestas de horario.

Paquetes principales:

Paquete	Uso
config	Configuración general, seguridad, seed de datos y manejo global de errores
controller	Endpoints REST
dto	Objetos de entrada y salida de la API
entity	Entidades JPA persistidas en BD
model	Enums y reglas de dominio
repository	Repositorios Spring Data JPA
security	JWT, filtros y usuario autenticado
service	Lógica de negocio

3.2. frontend/

Contiene la aplicación React. Sus responsabilidades principales son:

- Renderizar la interfaz de administración.
- Consumir la API REST.
- Guardar sesión/token en el navegador.
- Mostrar vistas de cursos, docentes, ambientes, horarios, usuarios y reglas.
- Aplicar validaciones de interfaz antes de enviar datos.

Archivos y carpetas importantes:

Ruta	Uso
src/App.jsx	Composición principal de rutas/vistas
src/components/	Componentes compartidos
src/contexts/	Contextos globales de autenticación y semestre
src/lib/api.js	Cliente HTTP contra el backend
src/lib/auth.js	Manejo de sesión local
src/lib/constants.js	Constantes de dominio
src/pages/	Pantallas principales
src/styles/	Estilos globales y por módulo

3.3. desktop/

Contiene el wrapper Electron. Su función es abrir la aplicación frontend como app de escritorio.

No reemplaza al backend ni a la base de datos. Para que la aplicación desktop funcione correctamente en desarrollo, normalmente deben estar levantados:

- PostgreSQL con Docker.
- Backend Spring Boot.
- Frontend Vite.
- Electron.

3.4. docs/

Contiene documentación técnica y funcional del proyecto. Algunos documentos clave:

Documento	Tema
ECONOMÍA-TOKENS.md	Reglas para trabajar eficientemente con agentes
GUÍA-DOCKER.md	Uso de Docker y PostgreSQL
USUARIOS-Y-PERSISTENCIA.md	Login, usuarios, JWT y persistencia
TURNOS-HORARIOS.md	Franjas horarias académicas
REGLAS-HORARIOS-CONFIGURABLES.md	Configuración de bloques horarios desde la app
CONDICIONES-DOCENTES.md	Reglas de asignación docente

VIABILIDAD-HORARIO-SIN-CONFLI CTOS.md	Análisis de capacidad y viabilidad
PLAN-ASIGNACIÓN-CURSOS-TURN OS.md	Plan técnico de asignación por curso/turno
PLAN-SINCRONIZACIÓN-DOCENTE -CURSO.md	Sincronización entre cursos y docentes

4. Requisitos de entorno

4.1. Herramientas principales

Herramienta	Uso
Git	Clonar y versionar el proyecto
Docker + Docker Compose	Levantar PostgreSQL
Java 21	Ejecutar el backend
Maven	Compilar y ejecutar Spring Boot
Node.js	Ejecutar frontend
npm o pnpm	Gestionar dependencias frontend/desktop
Electron	Ejecutar app de escritorio

4.2. Puertos usados

Servicio	Puerto
PostgreSQL en host	5433
PostgreSQL dentro del contenedor	5432
Backend Spring Boot	8081
Frontend Vite	5173

5. Base de datos

5.1. Motor

El proyecto usa PostgreSQL 16.

En desarrollo, PostgreSQL se levanta con Docker Compose usando el archivo `docker-compose.yml`.

Configuración principal:

services:

postgres:

image: postgres:16

container_name: schedule-postgres

environment:

POSTGRES_DB: schedule_db

POSTGRES_USER: schedule

POSTGRES_PASSWORD: schedule

ports:

- "5433:5432"

volumes:

- schedule_pg_data:/var/lib/postgresql/data

5.2. Datos de conexión

Parametro	Valor
Host	localhost
Puerto	5433
Base de datos	schedule_db
Usuario	schedule
Contraseña	schedule
Contenedor	schedule-postgres
Volumen	schedule_pg_data

5.3. Persistencia

Los datos persisten en el volumen Docker `schedule_pg_data`.

Esto significa que:

- `docker compose stop` detiene la BD sin borrar datos.
- `docker compose down` elimina el contenedor, pero conserva el volumen.
- `docker compose down -v` elimina también el volumen y borra los datos.

6. Configuración del backend

Archivo principal:

backend/src/main/resources/application.properties

Configuración actual:

server.port=8081

spring.datasource.url=\${DB_URL:jdbc:postgresql://localhost:5433/schedule_db}

spring.datasource.username=\${DB_USER:schedule}

spring.datasource.password=\${DB_PASSWORD:schedule}

spring.jpa.hibernate.ddl-auto=update

spring.jpa.show-sql=false

app.jwt.secret=\${JWT_SECRET:schedule-dev-jwt-secret-key-min-256-bits-long!!}

app.jwt.expiration-ms=86400000

6.1. Variables de entorno soportadas

Variable	Función	Valor por defecto
DB_URL	URL JDBC de PostgreSQL	jdbc:postgresql://localhost:5433/schedule_db
DB_USER	Usuario de BD	schedule
DB_PASSWORD	Contraseña de BD	schedule
JWT_SECRET	Llave usada para firmar JWT	Valor dev incluido

6.2. Persistencia JPA

El backend usa:

spring.jpa.hibernate.ddl-auto=update

Esto permite que Hibernate actualice el esquema de base de datos según las entidades JPA. Es práctico para desarrollo, pero para producción convendría usar migraciones controladas con Flyway o Liquibase.

7. Dependencias principales

7.1. Backend

Archivo:

backend/pom.xml

Tecnologías principales:

Dependencia	Uso
spring-boot-starter-web	API REST
spring-boot-starter-data-jpa	Persistencia JPA
spring-boot-starter-security	Seguridad y autorización
spring-boot-starter-validation	Validación de DTOs
postgresql	Driver JDBC PostgreSQL
jjwt-api, jjwt-impl, jjwt-jackson	Creacion y validación de JWT

Versiones relevantes:

Elemento	Versión
Java	21
Spring Boot	3.2.0
JJWT	0.12.6

7.2. Frontend

Archivo:

frontend/package.json

Dependencias principales:

Dependencia	Uso
react	UI
react-dom	Renderizado en navegador
vite	Dev server y build
tailwindcss	Estilos
lucide-react	Iconos
@base-ui/react	Componentes base
@radix-ui/react-slot	Composición de componentes
class-variance-authority, clsx, tailwind-merge	Utilidades de clases CSS

Scripts:

Comando	Uso
npm run dev o pnpm dev	Levantar Vite
npm run build o pnpm build	Compilar frontend
npm run preview o pnpm preview	Previsualizar build

7.3. Desktop

Archivo:

desktop/package.json

Dependencia principal:

Dependencia	Uso
electron	Ejecutar app de escritorio

Script:

pnpm start

8. Ejecución en desarrollo

8.1. Levantar PostgreSQL

Desde la raíz:

```
docker compose up -d
```

Verificar:

```
docker compose ps
```

8.2. Levantar backend

Desde backend/:

```
mvn spring-boot:run
```

El backend queda disponible en:

<http://localhost:8081>

8.3. Levantar frontend

Desde frontend/:

```
pnpm dev
```

o:

```
npm run dev
```

El frontend queda disponible normalmente en:

<http://localhost:5173>

8.4. Levantar desktop

Desde desktop/:

```
pnpm start
```


9. Autenticación y usuarios

9.1. Login inicial

El sistema crea un usuario administrador inicial:

Usuario	Contraseña	Rol
admin	admin123	ADMIN

9.2. Flujo de autenticación

- El usuario ingresa credenciales en la pantalla de login.
- El frontend llama a POST /api/auth/login.
- El backend valida credenciales contra PostgreSQL.
- Si son correctas, el backend devuelve datos del usuario y un JWT.
- El frontend guarda { user, token } en sessionStorage.
- Las peticiones protegidas incluyen:

Authorization: Bearer <token>

9.3. Roles

Rol	Permisos
ADMIN	Puede administrar usuarios y guardar reglas protegidas
USER	Puede acceder a funciones autenticadas sin permisos administrativos completos

9.4. Seguridad

El backend usa:

- Spring Security.
- JWT Bearer Token.
- BCrypt para contraseñas.
- Filtro JWT para autenticar requests.
- Restricciones por rol en rutas sensibles.

Archivos clave:

Archivo	Función
SecurityConfig.java	Configuración de seguridad
JwtService.java	Generación y validación de tokens
JwtAuthenticationFilter.java	Lectura del Bearer token
CustomUserDetailsService.java	Carga de usuarios desde BD
UserService.java	Lógica de usuarios y contraseñas

10. API REST

Todas las rutas bajo /api/**, excepto login, requieren autenticación según la configuración de seguridad.

10.1. Autenticación

Método	Ruta	Descripción
POST	/api/auth/login	Login y emisión de JWT

10.2. Usuarios

Base:

/api/users

Método	Ruta	Descripción
GET	/api/users	Listar usuarios
GET	/api/users/{id}	Obtener usuario
POST	/api/users	Crear usuario
PUT	/api/users/{id}	Editar usuario
PATCH	/api/users/{id}/status	Cambiar estado
DELETE	/api/users/{id}	Eliminar o desactivar

10.3. Docentes

Base:

/api/teachers

Método	Ruta	Descripción
GET	/api/teachers	Listar docentes
GET	/api/teachers/{id}	Obtener docente
POST	/api/teachers	Crear docente
PUT	/api/teachers/{id}	Editar docente
DELETE	/api/teachers/{id}	Eliminar docente

10.4. Cursos

Base:

/api/courses

Método	Ruta	Descripción
GET	/api/courses	Listar cursos
GET	/api/courses/{id}	Obtener curso
POST	/api/courses	Crear curso
PUT	/api/courses/{id}	Editar curso
DELETE	/api/courses/{id}	Eliminar curso

10.5. Ambientes o espacios

Base:

/api/spaces

Método	Ruta	Descripción
GET	/api/spaces	Listar ambientes
GET	/api/spaces/{id}	Obtener ambiente
POST	/api/spaces	Crear ambiente
PUT	/api/spaces/{id}	Editar ambiente
DELETE	/api/spaces/{id}	Eliminar ambiente

10.6. Jefes de práctica

Base:

/api/practice-heads

Método	Ruta	Descripción
GET	/api/practice-heads	Listar jefes de práctica
GET	/api/practice-heads/{id}	Obtener jefe de práctica
POST	/api/practice-heads	Crear jefe de práctica
PUT	/api/practice-heads/{id}	Editar jefe de práctica
DELETE	/api/practice-heads/{id}	Eliminar jefe de práctica

10.7. Reglas horarias

Base:

/api/schedule-settings

Método	Ruta	Descripción
GET	/api/schedule-settings	Obtener bloques horarios activos
PUT	/api/schedule-settings	Actualizar bloques horarios

10.8. Horarios

Base:

/api/schedules

Método	Ruta	Descripción
GET	/api/schedules	Obtener propuesta de horario
POST	/api/schedules/generate	Generar propuesta

10.9. Asignaciones curso-docente

Base:

/api/course-teacher-assignments

Método	Ruta	Descripción
PATCH	/api/course-teacher-assignments/{id}/weekday	Cambiar día de una asignación

10.10. Notificaciones

Base:

/api/notifications

Método	Ruta	Descripción
GET	/api/notifications	Listar notificaciones
GET	/api/notifications/logs	Listar logs

10.11. Prueba simple

Método	Ruta	Descripción
GET	/api/hello	Endpoint simple de prueba

11. Modelo de dominio

11.1. Usuarios

Entidad principal:

User

Campos conceptuales:

Campo	Descripción
id	Identificador
username	Nombre de usuario único
passwordHash	Contraseña hasheada
role	ADMIN o USER
enabled	Estado activo/inactivo

11.2. Docentes

Entidad principal:

Teacher

Tipos de docente:

Tipo	Descripción
NOMBRADO	Docente nombrado de la escuela
CONTRATADO	Docente contratado
ESTUDIOS_GENERALES	Docente asignado a cursos de estudios generales

Reglas relevantes:

- Nombrados y contratados solo deben dictar cursos de carrera.
- Estudios generales solo deben dictar cursos de estudios generales.
- Los nombrados tienen prioridad en el flujo operativo de asignación.
- Existe una regla institucional de máximo 17 docentes nombrados.

11.3. Cursos

Entidad principal:

Course

Los cursos pertenecen al plan de estudios EPIS 2024.

Clasificaciones relevantes:

Concepto	Ejemplos
Ciclo	I a X
Categoría	Carrera, estudios generales
Tipo	Teorico, laboratorio u otros según el modelo
Turno permitido	Mañana/tarde para I-VIII, noche para IX-X

11.4. Ambientes

Entidad principal:

Space

Representa aulas, laboratorios u otros ambientes donde pueden programarse cursos.

El sistema valida asignaciones para evitar conflictos de uso de ambientes.

11.5. Jefes de práctica

Entidad principal:

PracticeHead

Permite registrar y administrar encargados de práctica, especialmente asociados a laboratorios o cursos que requieren soporte práctico.

11.6. Bloques horarios

Entidad principal:

ScheduleBlockSetting

Representa las franjas del día académico:

- Desayuno.
- Turno mañana.
- Almuerzo.
- Turno tarde.
- Cena.
- Turno noche.

12. Reglas académicas de horarios

12.1. Dias lectivos

Las clases se programan de lunes a viernes.

No se consideran bloques lectivos sábado ni domingo.

12.2. Franjas horarias por defecto

Orden	Bloque	Código	Inicio	Fin
1	Desayuno	DESAYUNO	06:30	08:00
2	Turno mañana	MAÑANA	08:00	12:30
3	Almuerzo	ALMUERZO	12:30	14:00
4	Turno tarde	TARDE	14:00	17:00
5	Cena	CENA	17:00	17:15
6	Turno noche	NOCHE	17:15	22:30

12.3. Bloques lectivos y no lectivos

Bloques no lectivos:

- DESAYUNO
- ALMUERZO
- CENA

Bloques lectivos:

- MAÑANA
- TARDE
- NOCHE

12.4. Validación de bloques horarios

El sistema debe respetar:

- Exactamente 6 bloques.
- Orden fijo.
- Sin huecos entre bloques.
- Sin solapes.
- Duración mínima de 15 minutos por bloque.
- Inicio del desayuno no menor a 05:00.
- Fin de noche no mayor a 23:59.
- Formato HH:mm.

Ejemplo de encadenamiento correcto:

$\text{fin}(\text{DESAYUNO}) = \text{inicio}(\text{MAÑANA})$

$\text{fin}(\text{MAÑANA}) = \text{inicio}(\text{ALMUERZO})$

$\text{fin}(\text{ALMUERZO}) = \text{inicio}(\text{TARDE})$

$\text{fin}(\text{TARDE}) = \text{inicio}(\text{CENA})$

$\text{fin}(\text{CENA}) = \text{inicio}(\text{NOCHE})$

12.5. Configuración desde la app

La pantalla:

Reglas -> Horarios del día

permite administrar las franjas horarias.

Funciones esperadas:

- Ver timeline visual de lunes a viernes.
- Ajustar horas de inicio y fin.
- Guardar cambios.
- Restaurar valores por defecto.
- Persistir cambios en PostgreSQL.

Solo usuarios ADMIN deberían poder guardar cambios.

13. Reglas por ciclo

13.1. Ciclos I a VIII

Los ciclos I a VIII son diurnos.

Turnos permitidos:

- MAÑANA
- TARDE

No deben programarse en NOCHE.

13.2. Ciclos IX y X

Los ciclos IX y X son nocturnos.

Turno permitido:

- NOCHE

No deben programarse en MAÑANA ni TARDE.

13.3. Capacidad semanal

Capacidad por tipo de ciclo:

Tipo de ciclo	Turnos permitidos	Capacidad semanal
I-VIII	Mañana + tarde	37.5 h reales
IX-X	Noche	26.25 h reales

Carga semanal del plan:

Ciclo	Carga real semanal	Resultado
I	24.00 h	Viable
II	24.00 h	Viable
III	23.25 h	Viable
IV	24.00 h	Viable
V	25.50 h	Viable
VI	25.50 h	Viable
VII	25.50 h	Viable
VIII	25.50 h	Viable
IX	24.00 h	Viable
X	23.25 h	Viable

Conclusión: con los docentes, ambientes y asignaciones completas, la distribución es viable por capacidad.

14. Reglas de docentes

14.1. Tipos de docente

Tipo	Cursos permitidos
NOMBRADO	Cursos de carrera
CONTRATADO	Cursos de carrera
ESTUDIOS_GENERALES	Cursos de estudios generales

14.2. Flujo institucional de asignación

La asignación se entiende en tres fases:

Fase	Actor	Acción
1	Docentes nombrados	Eligen cursos y turnos de carrera
2	Estudios generales	Asigna docentes a cursos EG
3	Escuela EPIS	Contrata docentes para cubrir cursos faltantes

El sistema distingue tipos de docente y valida categorías, pero no automatiza completamente el flujo por fases.

14.3. Límite de nombrados

Existe una regla institucional:

Máximo 17 docentes nombrados

Según la documentación actual, esta condición debe respetarse operativamente. No necesariamente está impuesta de forma automática en todos los formularios.

14.4. Modalidades de asignación

Un docente puede asociarse a cursos por turno:

Modalidad	Descripción
Mañana y tarde	Mismo docente dicta el curso en ambos turnos
Solo mañana	Docente asignado a mañana
Solo tarde	Docente asignado a tarde
Turno noche	Docente asignado a noche

Para ciclos IX y X, la modalidad aplicable es noche.

15. Horarios y generación de propuestas

15.1. Fuente de verdad

La propuesta de horario se construye a partir de:

- Cursos registrados.
- Docentes asignados.
- Ambientes asignados.
- Turnos permitidos por ciclo.
- Subturnos cuando aplican.
- Bloques horarios activos.

15.2. Pantalla de horarios

Archivo principal:

frontend/src/pages/Horarios.jsx

Funciones descritas en la documentación:

- Consultar propuesta actual por ciclo.
- Mostrar matriz textual.
- Mostrar vista de color por franjas.
- Recalcular conforme se asignan docentes, cursos y ambientes.

15.3. Conflictos que debe evitar

El sistema busca evitar:

- Un docente en dos clases al mismo tiempo.
- Un ambiente en dos clases al mismo tiempo.
- Un ciclo con cursos superpuestos.
- Cursos fuera del turno permitido por ciclo.
- Cursos durante comida o bloques no lectivos.

15.4. Condiciones para viabilidad real

Para que el horario sea viable operativamente deben estar completos:

- Docentes de estudios generales.
- Docentes contratados necesarios.
- Asignaciones reales de docentes.
- Asignaciones reales de ambientes.
- Subturnos de laboratorios.
- Reglas horarias activas.

16. Frontend: pantallas principales

16.1. Login.jsx

Pantalla de inicio de sesión.

Responsabilidades:

- Capturar usuario y contraseña.
- Llamar al endpoint de login.
- Guardar sesión.
- Redirigir al área principal.

16.2. Dashboard.jsx

Pantalla inicial tras iniciar sesión.

Sirve como resumen o entrada al sistema.

16.3. Teachers.jsx

Gestión de docentes.

Funciones:

- Listar docentes.
- Crear docente.
- Editar docente.
- Eliminar docente.
- Asignar cursos a docentes.
- Mostrar cursos asociados.
- Respetar reglas por tipo de docente y curso.

16.4. Courses.jsx

Gestión de cursos.

Funciones:

- Listar cursos.
- Crear curso.
- Editar curso.
- Eliminar curso.
- Mostrar docentes asignados por turno.
- Sincronizar datos con asignaciones docente-curso.

16.5. Spaces.jsx

Gestión de ambientes.

Funciones:

- Listar ambientes.
- Crear ambiente.
- Editar ambiente.
- Eliminar ambiente.
- Asignar cursos a espacios.

- Controlar disponibilidad de ambientes.

16.6. PracticeHeads.jsx

Gestión de jefes de práctica.

Funciones:

- Registrar jefes de práctica.
- Editar información.
- Asociar responsabilidades de práctica/laboratorio según modelo.

16.7. Rules.jsx

Gestión de reglas.

Funciones:

- Ver horarios del día.
- Ajustar bloques horarios.
- Restaurar valores por defecto.
- Guardar configuración.

16.8. Horarios.jsx

Visualización/generación de horarios.

Funciones:

- Mostrar propuesta de horario.
- Organizar por ciclo.
- Ver matriz o vista de franjas.
- Consultar al backend la propuesta calculada.

16.9. Users.jsx

Gestión de usuarios.

Funciones:

- Listar usuarios.
- Crear usuario.
- Editar usuario.
- Activar/desactivar o eliminar.
- Restringir acceso a administradores.

17. Cliente HTTP del frontend

Archivo:

frontend/src/lib/api.js

Responsabilidades:

- Centralizar llamadas al backend.
- Adjuntar token JWT en requests protegidos.
- Manejar errores de autenticación.
- Ejecutar logout si el backend responde 401.

Flujo esperado:

UI -> api.js -> Backend REST -> PostgreSQL

18. Sesión frontend

Archivos clave:

frontend/src/lib/auth.js

frontend/src/contexts/AuthContext.jsx

Responsabilidades:

- Guardar datos de sesión en sessionStorage.
- Exponer usuario actual.
- Exponer funciones de login/logout.
- Controlar vistas visibles según autenticación y rol.

19. Sincronización curso-docente

La relación entre docentes y cursos se maneja mediante:

CourseTeacherAssignment

Conceptualmente:

- Un docente puede tener cursos asignados.
- Un curso puede mostrar docentes asignados por turno.
- La base de datos es la fuente de verdad.
- El frontend debe refrescar vistas para reflejar cambios cruzados.

Problema documentado:

- Al guardar desde Teachers.jsx, Courses.jsx puede necesitar recargar datos para mostrar el docente asignado.
- Al guardar desde Courses.jsx, Teachers.jsx puede necesitar recargar datos para mostrar el curso asignado.

Solución sugerida en documentación:

- Usar eventos de ventana como teachers-updated y courses-updated.
- Cada página escucha el evento que le corresponde y refresca datos.

20. Plan de estudios y carga horaria

El proyecto incluye información del plan EPIS 2024.

Resumen global:

Tipo de estudio	Cursos	H. académicas	Créditos	H. reales
Estudios Generales	12	59	41	44.25
Estudios Específicos	13	60	44	45.00
Estudios de Especialidad	35	199	134	149.25
Electivos	2	8	6	6.00
Total	62	326	225	244.50

Regla de conversion:

1 hora académica = 45 minutos

21. Documentación existente recomendada

Para entender o mantener el sistema, revisar en este orden:

- README.md
- docs/GUÍA-DOCKER.md
- docs/USUARIOS-Y-PERSISTENCIA.md
- docs/TURNOS-HORARIOS.md
- docs/REGLAS-HORARIOS-CONFIGURABLES.md
- docs/CONDICIONES-DOCENTES.md
- docs/VIABILIDAD-HORARIO-SIN-CONFLICTOS.md
- docs/PLAN-ASIGNACIÓN-CURSOS-TURNOS.md
- docs/PLAN-SINCRONIZACIÓN-DOCENTE-CURSO.md

Este documento consolida los puntos principales, pero los documentos especializados siguen siendo la referencia más detallada para cada tema concreto.

22. Flujo recomendado para desarrollo

22.1. Antes de modificar

- Identificar el módulo afectado.
- Buscar archivos relevantes con rg.
- Leer solo los archivos necesarios.
- Confirmar si el cambio afecta backend, frontend o ambos.
- Editar con parches pequenos.

22.2. Si el cambio es de backend

Revisar normalmente:

- Controller relacionado.
- DTOs de entrada/salida.
- Service con reglas de negocio.
- Entity si cambia persistencia.
- Repository si requiere consultas.
- Security si afecta permisos.

22.3. Si el cambio es de frontend

Revisar normalmente:

- Página en frontend/src/pages/.
- Componentes usados por esa página.
- Cliente API.
- Constantes de dominio.
- Contextos si afecta sesión o estado global.
- Estilos del módulo.

22.4. Si el cambio afecta reglas académicas

Revisar:

- docs/CONDICIONES-DOCENTES.md
- docs/TURNOS-HORARIOS.md
- docs/REGLAS-HORARIOS-CONFIGURABLES.md
- CourseCycleRules.java
- Services correspondientes.
- frontend/src/lib/constants.js

23. Reglas para agentes en este repositorio

El archivo AGENTS.md define cómo debe trabajar un agente en el proyecto.

Reglas principales:

- Responder siempre en español, salvo nombres de archivos o comandos.
- Mantener respuestas breves y directas cuando la tarea sea concreta.
- Para cambios de código, mostrar solo archivos editados, fragmentos mínimos y resumen corto.
- Evitar builds, instalaciones o dependencias nuevas sin permiso.
- Leer el archivo actual antes de modificarlo.
- Buscar antes de leer grandes partes del proyecto.
- Editar solo bloques específicos.
- No tocar código fuera del área solicitada.

Guía ampliada:

docs/ECONOMÍA-TOKENS.md

24. Problemas frecuentes

24.1. PostgreSQL no levanta

Verificar:

`docker compose ps`

`docker compose logs postgres --tail 20`

Posibles causas:

- Docker no está iniciado.
- Puerto 5433 ocupado.
- Volumen corrupto o configuración previa incompatible.

24.2. Backend no conecta a BD

Verificar:

- PostgreSQL está Up.
- `application.properties` apunta a `localhost:5433`.
- Usuario y contraseña son `schedule`.
- La base se llama `schedule_db`.

24.3. Puerto 8081 ocupado

Significa que ya existe un backend ejecutándose o algún proceso usa el puerto.

Solución:

- Cerrar la terminal anterior.
- Identificar y detener el proceso que usa el puerto.

24.4. Login falla

Revisar:

- Backend encendido.
- PostgreSQL encendido.
- Usuario inicial `admin`.
- Contraseña `admin123`.
- Seed inicial ejecutado correctamente.

24.5. Frontend no muestra datos actualizados

Posibles causas:

- Token expirado o inválido.
- Backend apagado.
- Cambios guardados desde otra pantalla sin refresco de datos.
- Error de CORS o API base incorrecta.

25. Consideraciones para producción

Antes de producción convendría:

- Cambiar JWT_SECRET.
- No usar credenciales por defecto.
- Configurar PostgreSQL en servidor central.
- Deshabilitar secretos hardcodeados.
- Usar migraciones con Flyway o Liquibase.
- Configurar CORS para dominio real.
- Construir frontend con pnpm build o npm run build.
- Ajustar Electron para cargar frontend/dist/index.html.
- Definir backups de PostgreSQL.
- Definir políticas de roles y usuarios.

26. Resumen ejecutivo final

Schedule es una aplicación académica full-stack para administrar horarios, docentes, cursos, ambientes y usuarios. El backend Spring Boot protege la API con JWT, aplica reglas de negocio y persiste en PostgreSQL. El frontend React consume esa API y ofrece pantallas para operar el sistema. Electron permite abrir la interfaz como app de escritorio.

Las reglas académicas centrales son:

- Clases de lunes a viernes.
- Comidas bloqueadas: desayuno, almuerzo y cena.
- Ciclos I-VIII en mañana/tarde.
- Ciclos IX-X en noche.
- Docentes nombrados y contratados para cursos de carrera.
- Docentes de estudios generales para cursos EG.
- Horarios configurables desde la pantalla de reglas.
- Propuesta de horario viable si se completan docentes, ambientes y asignaciones.

Este documento sirve como mapa maestro del proyecto y debe mantenerse alineado con los documentos especializados de docs/.

Parte 2: Explicación del funcionamiento

Explicación de como funciona Schedule

Portada

Proyecto: Schedule Desktop App

Documento: Explicación del funcionamiento del sistema

Enfoque: Presentación clara para explicar qué hace cada parte y cómo se conectan

Tecnologías: React, Vite, Spring Boot, PostgreSQL, Docker y Electron

1. Explicación general

Schedule es una aplicación para organizar información académica y apoyar la elaboración de horarios.

El sistema permite registrar usuarios, docentes, cursos, ambientes, jefes de práctica y reglas horarias. Con esa información, la aplicación puede consultar, validar y construir propuestas de horario respetando condiciones académicas.

La idea principal es que el usuario no trabaje directamente con archivos ni con la base de datos. El usuario usa una interfaz visual. Esa interfaz se comunica con un backend, y el backend se encarga de guardar, validar y devolver la información.

En terminos simples:

El usuario usa la pantalla.

La pantalla pide datos al backend.

El backend consulta o guarda en la base de datos.

La base de datos conserva la información.

El backend responde.

La pantalla muestra el resultado.

2. Partes principales del sistema

Schedule está formado por cuatro partes importantes.

Parte	Qué es	Para qué sirve
Frontend	Interfaz hecha con React	Permite que el usuario use el sistema visualmente
Backend	API hecha con Spring Boot	Procesa solicitudes, valida reglas y conecta con la base de datos
Base de datos	PostgreSQL	Guarda usuarios, docentes, cursos, ambientes, reglas y asignaciones
Desktop	Electron	Abre la aplicación como programa de escritorio

Estas partes trabajan juntas, pero cada una cumple una función distinta.

3. Cómo funciona cuando se abre la aplicación

Para usar Schedule en desarrollo se levantan primero los servicios necesarios.

Primero se inicia PostgreSQL con Docker. PostgreSQL es donde se guardan los datos.

Luego se inicia el backend Spring Boot. El backend queda esperando peticiones en el puerto 8081.

Después se inicia el frontend React. El frontend muestra las pantallas del sistema.

Finalmente, si se quiere usar como aplicación de escritorio, se inicia Electron. Electron abre una ventana y carga el frontend.

El orden normal es:

1. PostgreSQL
2. Backend
3. Frontend
4. Electron

Si la base de datos no está preñida, el backend no puede guardar ni consultar información. Si el backend no está preñido, el frontend no puede iniciar sesión ni cargar datos.

4. Cómo funciona el inicio de sesión

Cuando el usuario abre la aplicación, primero aparece la pantalla de login.

El usuario escribe su nombre de usuario y contraseña. La pantalla no decide por sí sola si el usuario puede entrar. Envía esos datos al backend.

El backend revisa si el usuario existe en PostgreSQL y si la contraseña es correcta. Las contraseñas no se guardan en texto normal, sino como hash usando BCrypt.

Si todo es correcto, el backend genera un token JWT. Ese token funciona como una credencial temporal para que el usuario pueda usar las demás partes del sistema.

Flujo:

Usuario escribe credenciales

- > Frontend envía login
- > Backend valida usuario y contraseña
- > PostgreSQL confirma datos
- > Backend genera token JWT
- > Frontend guarda sesión
- > Usuario entra al sistema

El usuario inicial es:

Usuario	Contraseña	Rol
admin	admin123	ADMIN

5. Cómo funciona la seguridad

Schedule usa roles para controlar permisos.

Hay dos roles principales:

Rol	Que puede hacer
ADMIN	Puede administrar usuarios y modificar reglas protegidas
USER	Puede usar funciones del sistema sin administrar cuentas

Cuando el frontend hace una petición protegida, envía el token JWT al backend.

Authorization: Bearer <token>

El backend revisa ese token. Si es válido, permite la operación. Si no es válido, responde con error.

Esto evita que cualquier persona pueda entrar directamente a rutas internas del sistema.

6. Como viajan los datos

Los datos viajan principalmente en formato JSON.

Ejemplo: cuando se crea un docente, el frontend envia información como nombre, tipo de docente y otros campos. El backend recibe esos datos, los valida y los guarda.

Flujo general:

Pantalla React

- > api.js
- > Backend REST
- > Servicio de negocio
- > Repositorio JPA
- > PostgreSQL

Luego la respuesta regresa:

PostgreSQL

- > Repositorio
- > Servicio
- > Controller
- > JSON
- > Frontend
- > Pantalla actualizada

El archivo frontend/src/lib/api.js es importante porque centraliza las llamadas al backend. Así las pantallas no repiten la misma lógica de conexión.

7. Como se guardan los datos

Los datos se guardan en PostgreSQL.

El frontend no guarda información definitiva. Puede tener datos temporales en pantalla, pero la fuente real es la base de datos.

Ejemplo:

Si se crea un curso y el backend confirma el guardado, entonces el curso queda registrado en PostgreSQL.

Si solo se escribe en el formulario pero no se guarda, el dato aún no existe realmente en el sistema.

Esto es importante porque permite cerrar y volver a abrir la aplicación sin perder la información.

8. Cómo funciona la gestión de usuarios

La pantalla de usuarios permite que un administrador cree, edite o desactive cuentas.

Cuando se crea un usuario:

1. El admin llena el formulario.
2. El frontend envía los datos al backend.
3. El backend valida que el username no exista.
4. El backend convierte la contraseña en hash.
5. El backend guarda el usuario en PostgreSQL.
6. El frontend actualiza la tabla de usuarios.

El sistema no debería guardar contraseñas en texto plano. Por seguridad, guarda un hash.

9. Cómo funciona la gestión de docentes

Los docentes se registran con un tipo.

Tipos principales:

Tipo	Explicación
NOMBRADO	Docente estable de la escuela
CONTRATADO	Docente que cubre cursos pendientes
ESTUDIOS_GENERALES	Docente para cursos de estudios generales

El tipo de docente es importante porque no todos pueden dictar cualquier curso.

Regla principal:

Nombrados y contratados -> cursos de carrera.

Estudios generales -> cursos de estudios generales.

Cuando se asigna un docente a un curso, el backend revisa si esa relación es válida.

10. Cómo funciona la gestión de cursos

Los cursos pertenecen al plan de estudios.

Cada curso tiene datos como:

- nombre;
- código;
- ciclo;
- categoría;
- horas;
- créditos;
- docentes asignados;
- turnos permitidos.

El ciclo del curso ayuda a decidir donde puede ubicarse en el horario.

Regla:

Ciclos	Turno permitido
I a VIII	Mañana y tarde
IX y X	Noche

Por ejemplo, un curso de ciclo IX no debería programarse en la mañana.

11. Cómo funciona la gestión de ambientes

Los ambientes son los espacios físicos donde se dictan clases.

Pueden ser:

- aulas;
- laboratorios;
- salas;
- otros espacios académicos.

El sistema debe evitar que un ambiente sea usado por dos cursos al mismo tiempo.

Ejemplo de conflicto:

Aula 101 - Lunes 08:00 a 10:00 - Curso A

Aula 101 - Lunes 09:00 a 11:00 - Curso B

Ese caso no debe permitirse porque se cruzan en el mismo ambiente.

12. Cómo funcionan las reglas horarias

Schedule divide el día académico en seis bloques.

Bloque	Tipo	Función
Desayuno	No lectivo	Pausa
Mañana	Lectivo	Clases
Almuerzo	No lectivo	Pausa
Tarde	Lectivo	Clases
Cena	No lectivo	Pausa
Noche	Lectivo	Clases

Los bloques no lectivos no deben tener clases.

Los bloques lectivos son los que se usan para ubicar cursos.

La pantalla de reglas permite modificar los horarios del día académico. Cuando se guarda, el backend valida que no haya huecos, solapes ni horarios inválidos.

13. Cómo funciona la asignación docente-curso

Una asignación docente-curso indica que un docente dicta un curso en un turno determinado.

Ejemplo:

Docente: Juan Perez

Curso: Base de Datos

Turno: Mañana

Esa relación se guarda para que luego el horario pueda saber:

- qué docente dicta el curso;
- en qué turno debe ubicarse;
- si el docente ya está ocupado;
- si el curso ya está cubierto.

La asignación conecta dos pantallas: docentes y cursos. Si se cambia desde una pantalla, la otra debe actualizarse para mostrar la información correcta.

14. Cómo funciona la generación de horarios

La generación de horarios depende de datos previos.

Antes de generar una propuesta, el sistema necesita:

- cursos registrados;
- docentes registrados;
- docentes asignados a cursos;
- ambientes disponibles;
- reglas horarias configuradas;
- turnos permitidos por ciclo.

Con esa información, el backend intenta construir una propuesta evitando conflictos.

El flujo es:

Usuario abre Horarios

- > Frontend pide una propuesta
- > Backend consulta cursos, docentes, ambientes y reglas
- > Backend aplica restricciones
- > Backend genera resultado
- > Frontend muestra el horario

La propuesta debe evitar:

- docente duplicado en la misma hora;
- ambiente duplicado en la misma hora;
- dos cursos del mismo ciclo al mismo tiempo;
- cursos en bloques de comida;
- cursos fuera del turno permitido.

15. Cómo funciona Docker en este proyecto

Docker se usa para levantar PostgreSQL fácilmente.

No significa que toda la aplicación esté dentro de Docker. En este proyecto, Docker se usa principalmente para la base de datos.

Ventaja:

No es necesario instalar PostgreSQL manualmente.

Docker levanta la base de datos con la configuración ya definida.

El comando principal es:

```
docker compose up -d
```

Los datos se guardan en un volumen. Por eso no se pierden al apagar el contenedor.

16. Cómo funciona Electron

Electron permite que la aplicación se vea como un programa de escritorio.

Electron abre una ventana y carga el frontend.

Pero Electron no es el backend y no es la base de datos.

Función real de Electron:

Abrir una ventana de escritorio

- > cargar el frontend

- > permitir usar la app como programa local

Para que Electron funcione bien en desarrollo, también deben estar funcionando PostgreSQL, backend y frontend.

17. Ejemplo completo explicado

Supongamos que el administrador quiere crear un curso y luego verlo en la lista.

Paso a paso:

1. El administrador inicia sesión.
2. El backend valida su usuario.
3. El frontend guarda el token.
4. El administrador entra a Cursos.
5. El frontend pide la lista de cursos.
6. El backend consulta PostgreSQL.
7. La pantalla muestra los cursos existentes.
8. El administrador llena el formulario de nuevo curso.
9. El frontend envia el curso al backend.
10. El backend valida los datos.
11. El backend guarda el curso en PostgreSQL.
12. El backend responde que se guardó correctamente.
13. El frontend refresca la lista.
14. El nuevo curso aparece en pantalla.

Este mismo patron se repite en usuarios, docentes, ambientes y otras entidades.

18. Explicación final para exposición

Schedule funciona como una aplicación académica completa porque separa responsabilidades.

El frontend se encarga de la experiencia visual. El backend se encarga de la lógica, seguridad y validaciones. PostgreSQL guarda los datos. Docker facilita levantar la base de datos. Electron permite abrir la aplicación como escritorio.

La comunicación entre partes permite que el sistema no sea solo una maqueta. Cada acción importante pasa por el backend y queda registrada en la base de datos.

La parte central del proyecto es la aplicación de reglas académicas: los cursos dependen de ciclos, los ciclos dependen de turnos, los docentes dependen de tipos, los ambientes no deben cruzarse y los horarios deben respetar bloques lectivos y no lectivos.

Por eso, Schedule ayuda a organizar la programación académica de forma más controlada, evitando errores comunes y manteniendo la información centralizada.

Parte 3: Funcionamiento interno

Funcionamiento interno del proyecto Schedule

Tema del documento: Como funcionan las partes del sistema y cómo se comunican entre sí

Tipo de sistema: Aplicación académica full-stack para gestión de horarios

Componentes principales: Spring Boot, React, Vite, Electron, PostgreSQL y Docker

1. Idea general del funcionamiento

Schedule funciona como un sistema dividido en capas. Cada capa cumple una responsabilidad clara y se comunica con las demás mediante interfaces definidas.

La aplicación no guarda los datos directamente en el frontend. El usuario interactúa con pantallas React, esas pantallas llaman a una API REST del backend, el backend valida la solicitud, aplica reglas de negocio y finalmente guarda o consulta información en PostgreSQL.

El funcionamiento general puede resumirse así:

Usuario

- > Frontend React
- > Cliente HTTP api.js
- > Backend Spring Boot
- > Servicios de negocio
- > Repositorios JPA
- > PostgreSQL

Cuando se usa la versión de escritorio, Electron actúa como una ventana que muestra el frontend. Electron no reemplaza al backend ni a la base de datos.

Electron

- > Carga la interfaz React
- > React consume el backend
- > Backend usa PostgreSQL

2. Como se inicia el sistema

Para que todo funcione en desarrollo deben estar activos varios componentes.

2.1. Base de datos

Primero se levanta PostgreSQL con Docker Compose:

```
docker compose up -d
```

Esto crea o inicia un contenedor llamado schedule-postgres, usando PostgreSQL 16. La base de datos queda disponible en el puerto 5433 del equipo.

Datos principales:

Elemento	Valor
Base de datos	schedule_db
Usuario	schedule
Contraseña	schedule

Puerto host	5433
Puerto interno	5432
Contenedor	schedule-postgres
Volumen	schedule_pg_data

El volumen schedule_pg_data permite conservar la información aunque el contenedor se apague.

2.2. Backend

Luego se levanta el backend:

```
cd backend
```

```
mvn spring-boot:run
```

El backend queda escuchando en:

http://localhost:8081

Al iniciar, Spring Boot:

- carga la configuración de application.properties;
- abre conexión con PostgreSQL;
- registra controladores REST;
- configura Spring Security;
- prepara el filtro JWT;
- carga repositorios JPA;
- crea o actualiza tablas con Hibernate;
- inserta datos iniciales si corresponde.

2.3. Frontend

Después se levanta el frontend:

```
cd frontend
```

```
pnpm dev
```

o:

```
cd frontend
```

```
npm run dev
```

Vite normalmente expone la interfaz en:

http://localhost:5173

El frontend no contiene la base de datos. Su función es mostrar pantallas, capturar acciones del usuario y comunicarse con el backend.

2.4. Desktop

Finalmente, si se desea usar la versión de escritorio:

```
cd desktop
```

```
pnpm start
```

Electron abre una ventana de escritorio y carga la aplicación frontend.

3. Como se comunican las capas

La comunicación principal ocurre por HTTP.

El frontend llama a rutas como:

GET /api/users

POST /api/auth/login

GET /api/teachers

PUT /api/courses/{id}

POST /api/schedules/generate

El backend responde con JSON. El frontend transforma ese JSON en tablas, formularios, tarjetas, opciones o grillas de horario.

Ejemplo de flujo:

Usuario presiona "Guardar docente"

- > Teachers.jsx arma los datos del formulario
- > api.js envia POST /api/teachers
- > TeacherController recibe la petición
- > TeacherService valida reglas
- > TeacherRepository guarda en PostgreSQL
- > Backend responde JSON
- > Frontend refresca la lista de docentes

4. Funcionamiento del login

El login es el primer flujo importante porque protege el resto de la aplicación.

4.1. Usuario inicial

El sistema crea un administrador inicial:

Usuario	Contraseña	Rol
admin	admin123	ADMIN

Este usuario permite entrar por primera vez y crear otras cuentas desde la pantalla de usuarios.

4.2. Flujo paso a paso

- El usuario escribe nombre de usuario y contraseña en Login.jsx.
- El frontend llama a POST /api/auth/login.
- AuthController recibe la solicitud.
- El backend busca el usuario en PostgreSQL.
- Spring Security compara la contraseña ingresada con el hash BCrypt guardado.
- Si las credenciales son validas, JwtService genera un token JWT.
- El backend devuelve el usuario y el token.
- El frontend guarda la sesión en sessionStorage.
- La aplicación muestra las pantallas internas.

Resumen visual:

Login.jsx

-> api.js
-> POST /api/auth/login
-> AuthController
-> CustomUserDetailsService
-> UserRepository
-> PostgreSQL
-> JwtService
-> token
-> sessionStorage

4.3. Que guarda el frontend

El frontend guarda una estructura parecida a:

```
{  
  "user": {  
    "username": "admin",  
    "role": "ADMIN"
```



```
},  
  "token": "jwt..."  
}
```

Esto se guarda en sessionStorage, no directamente en la base de datos.

4.4. Por que se usa JWT

El JWT permite que el backend reconozca al usuario en cada petición sin pedir usuario y contraseña otra vez.

Cada petición protegida incluye:

Authorization: Bearer <token>

El backend valida ese token antes de permitir acceso.

5. Funcionamiento de la autorización

No todos los usuarios pueden hacer lo mismo.

El sistema maneja roles:

Rol	Función
ADMIN	Puede administrar usuarios y guardar configuraciones protegidas
USER	Puede usar funciones autenticadas sin control administrativo completo

Ejemplo:

- un ADMIN puede entrar a la pantalla Usuarios;
- un USER no debería administrar cuentas;
- algunas reglas del sistema solo deberían guardarse desde una cuenta administradora.

El control ocurre en dos lugares:

- En el frontend, ocultando o limitando opciones según el rol.
- En el backend, rechazando peticiones no autorizadas.

El backend es la capa más importante para seguridad, porque el frontend puede ser manipulado por un usuario avanzado. Por eso las rutas sensibles también se protegen desde Spring Security.

6. Funcionamiento del cliente HTTP

El archivo:

frontend/src/lib/api.js

centraliza las llamadas al backend.

Sus responsabilidades son:

- construir URLs hacia la API;
- enviar datos en formato JSON;
- agregar el token JWT cuando existe sesión;
- detectar errores HTTP;
- cerrar sesión si el backend responde 401;
- evitar repetir código HTTP en todas las pantallas.

Flujo normal:

Componente React

-> llama función de api.js

-> api.js agrega headers

-> fetch al backend

-> backend responde JSON

-> api.js devuelve datos al componente

Esto hace que las pantallas no tengan que saber todos los detalles técnicos del HTTP.

7. Funcionamiento de la persistencia

La persistencia significa que los datos no se pierden al cerrar la interfaz.

Schedule usa PostgreSQL como base de datos real. El backend se comunica con PostgreSQL mediante JPA.

7.1. Papel de las entidades

Las entidades representan tablas de la base de datos.

Ejemplos conceptuales:

Entidad	Que representa
User	Cuentas del sistema
Teacher	Docentes
Course	Cursos
Space	Ambientes
PracticeHead	Jefes de práctica
ScheduleBlockSetting	Bloques horarios configurables
CourseTeacherAssignment	Relación entre cursos y docentes

7.2. Papel de los repositorios

Los repositorios son interfaces que permiten consultar y guardar datos sin escribir SQL manual en cada operación.

Ejemplo conceptual:

TeacherService

-> TeacherRepository

-> PostgreSQL

7.3. Papel de los servicios

Los servicios contienen la lógica de negocio.

El controlador no debería decidir reglas complejas. Su trabajo principal es recibir peticiones y devolver respuestas.

El servicio decide cosas como:

- si un docente puede asignarse a cierto curso;
- si un usuario puede crearse;
- si una contraseña debe actualizarse;
- si un bloque horario es válido;
- si una asignación genera conflicto.

8. Funcionamiento de los CRUD

CRUD significa:

Letra	Operación	Significado
C	Create	Crear
R	Read	Leer/Listar
U	Update	Actualizar
D	Delete	Eliminar o desactivar

Schedule usa CRUD para varias entidades: usuarios, docentes, cursos, ambientes y jefes de práctica.

8.1. Crear

Ejemplo: crear un docente.

Usuario llena formulario

- > Frontend valida campos basicos
- > Frontend envia POST /api/teachers
- > Backend valida reglas
- > Backend guarda en PostgreSQL
- > Frontend actualiza tabla

8.2. Leer o listar

Ejemplo: abrir la pantalla de cursos.

Courses.jsx se monta

- > llama GET /api/courses
- > backend consulta CourseRepository
- > PostgreSQL devuelve cursos
- > frontend muestra lista

8.3. Actualizar

Ejemplo: editar un ambiente.

Usuario selecciona ambiente

- > formulario carga datos existentes
- > usuario modifica información
- > frontend envia PUT /api/spaces/{id}
- > backend actualiza registro
- > frontend refresca la vista

8.4. Eliminar o desactivar

Dependiendo de la entidad, eliminar puede significar borrar el registro o marcarlo como inactivo.

En usuarios, es más seguro desactivar que borrar, porque conserva trazabilidad.

9. Funcionamiento de usuarios

La pantalla Users.jsx permite administrar cuentas.

Flujo de usuario administrador:

- Inicia sesión como ADMIN.
- Entra a la pantalla Usuarios.
- Consulta la lista de cuentas.
- Crea un nuevo usuario indicando username, contraseña y rol.
- Edita usuarios existentes si es necesario.
- Desactiva cuentas que ya no deben ingresar.

El backend guarda:

- username;
- hash de contraseña;
- rol;
- estado activo/inactivo.

La contraseña nunca debe guardarse en texto plano. Se guarda como hash BCrypt.

10. Funcionamiento de docentes

La pantalla de docentes permite registrar y administrar profesores.

Un docente tiene un tipo:

Tipo	Uso
NOMBRADO	Docente estable de la escuela
CONTRATADO	Docente contratado para cubrir cursos
ESTUDIOS_GENERALES	Docente asignado a cursos de estudios generales

El sistema usa ese tipo para validar que el docente sea compatible con los cursos.

Regla general:

- docentes NOMBRADO y CONTRATADO dictan cursos de carrera;
- docentes ESTUDIOS_GENERALES dictan cursos de estudios generales.

Flujo de asignación:

Usuario selecciona docente

- > elige curso y turno
- > frontend envía asignación
- > backend valida compatibilidad
- > backend guarda relación curso-docente
- > frontend actualiza docentes y cursos

11. Funcionamiento de cursos

Los cursos pertenecen al plan de estudios EPIS 2024.

Cada curso puede tener datos como:

- código;
- nombre;
- ciclo;
- categoría;
- horas;
- créditos;
- turnos permitidos;
- docentes asignados.

La pantalla de cursos permite crear, editar, eliminar y consultar cursos. También muestra relaciones con docentes.

El ciclo del curso es importante porque determina en qué turno puede programarse:

Ciclos	Turnos permitidos
--------	-------------------

I a VIII	Mañana y tarde
IX y X	Noche

Esto evita que cursos de los últimos ciclos se mezclen con horarios diurnos si la regla institucional indica que deben ser nocturnos.

12. Funcionamiento de ambientes

Los ambientes representan aulas, laboratorios u otros espacios físicos.

La pantalla de ambientes permite:

- registrar espacios;
- editar información;
- eliminar ambientes;
- asignar cursos;
- controlar disponibilidad.

En una propuesta de horario, un ambiente no debería usarse en dos clases al mismo tiempo.

Ejemplo de conflicto:

Laboratorio 1

Lunes 08:00 - 10:00

Curso A

Laboratorio 1

Lunes 09:00 - 11:00

Curso B

Ese caso debe evitarse porque el mismo ambiente queda ocupado simultaneamente.

13. Funcionamiento de jefes de práctica

Los jefes de práctica sirven para registrar personal asociado a sesiones prácticas o laboratorios.

Su gestión permite:

- registrar datos del jefe de práctica;
- actualizar información;
- relacionar responsabilidades prácticas;
- mantener información separada de los docentes principales.

Esta separación ayuda cuando un curso tiene teoría con un docente y práctica con otro responsable.

14. Funcionamiento de reglas horarias

Las reglas horarias definen el día académico.

El sistema usa seis bloques:

Orden	Bloque	Código	Inicio	Fin	Tipo
1	Desayuno	DESAYUN O	06:30	08:00	No lectivo
2	Mañana	MAÑANA	08:00	12:30	Lectivo
3	Almuerzo	ALMUERZ O	12:30	14:00	No lectivo
4	Tarde	TARDE	14:00	17:00	Lectivo
5	Cena	CENA	17:00	17:15	No lectivo
6	Noche	NOCHE	17:15	22:30	Lectivo

Los bloques no lectivos son pausas. No se deben programar clases durante desayuno, almuerzo o cena.

14.1. Como se editan

En la pantalla:

Reglas -> Horarios del día

un administrador puede modificar horarios.

Flujo:

Rules.jsx carga GET /api/schedule-settings

- > muestra bloques actuales
- > usuario cambia horas
- > frontend valida formato
- > backend valida reglas completas
- > PUT /api/schedule-settings
- > PostgreSQL guarda configuración activa

14.2. Validaciones

El sistema debe respetar:

- exactamente seis bloques;
- orden fijo;
- sin huecos entre bloques;
- sin solapes;
- duración mínima de 15 minutos;
- formato HH:mm;
- inicio no menor a 05:00;
- fin no mayor a 23:59.

15. Funcionamiento de turnos por ciclo

La regla académica principal es:

Ciclo	Funcionamiento
I a VIII	Se programan en mañana y tarde
IX y X	Se programan en noche

Esto organiza los cursos por nivel académico y evita que los últimos ciclos se mezclen con turnos que no les corresponden.

Ejemplo:

Curso de ciclo III

-> puede ir en MAÑANA o TARDE

-> no debe ir en NOCHE

Curso de ciclo IX

-> debe ir en NOCHE

-> no debe ir en MAÑANA ni TARDE

16. Funcionamiento de asignaciones curso-docente

La relación entre cursos y docentes no es simplemente texto en pantalla. Debe guardarse como una relación persistente.

Conceptualmente:

Course

-> CourseTeacherAssignment

-> Teacher

Esta relación permite saber:

- qué docente dicta un curso;
- en qué turno lo dicta;
- si el mismo docente cubre mañana y tarde;
- si un curso nocturno ya tiene docente;
- qué cursos aparecen asociados a cada docente.

Cuando se modifica una asignación desde docentes, también debe reflejarse en cursos. Cuando se modifica desde cursos, también debe reflejarse en docentes.

Por eso la documentación propone refrescar vistas mediante eventos como:

teachers-updated

courses-updated

17. Funcionamiento de horarios

La pantalla de horarios consulta o genera una propuesta organizada por ciclo.

Para construir una propuesta se necesita:

- lista de cursos;
- docentes asignados;
- ambientes asignados;
- ciclo de cada curso;
- reglas de turnos;
- bloques horarios activos;
- restricciones de conflictos.

Flujo general:

Usuario entra a Horarios

- > Horarios.jsx solicita propuesta
- > backend consulta cursos, docentes, ambientes y reglas
- > servicio de horarios calcula ubicaciones
- > backend devuelve matriz o lista de bloques
- > frontend renderiza propuesta

17.1. Conflictos que se deben evitar

El sistema debe evitar:

- un docente en dos clases a la misma hora;
- un ambiente ocupado por dos cursos al mismo tiempo;
- un ciclo con dos cursos superpuestos;
- cursos de ciclo I-VIII en turno noche;
- cursos de ciclo IX-X en mañana o tarde;
- clases en bloques de comida;
- asignaciones sin docente cuando el curso ya debe programarse;
- asignaciones sin ambiente cuando el curso requiere espacio.

17.2. Por que la propuesta depende de datos completos

El horario no puede ser completamente confiable si faltan datos.

Ejemplos:

- si un curso no tiene docente, no se puede validar conflicto docente;
- si un curso no tiene ambiente, no se puede validar conflicto de aula;
- si no están configurados los bloques horarios, no se puede ubicar correctamente;
- si faltan cursos del plan, la carga horaria queda incompleta.

18. Funcionamiento visual del frontend

El frontend está organizado por pantallas.

Pantalla	Función
Login.jsx	Iniciar sesión
Dashboard.jsx	Entrada principal
Users.jsx	Administrar usuarios
Teachers.jsx	Administrar docentes
Courses.jsx	Administrar cursos
Spaces.jsx	Administrar ambientes
PracticeHeads.jsx	Administrar jefes de práctica
Rules.jsx	Configurar reglas horarias
Horarios.jsx	Ver o generar horarios

Cada pantalla combina:

- estado React;
- formularios;
- tablas;
- llamadas a api.js;
- validaciones de interfaz;
- mensajes de error o éxito;
- renderizado de datos devueltos por el backend.

19. Funcionamiento del backend por capas

El backend sigue una estructura por responsabilidades.

Capa	Función
Controller	Recibe HTTP y devuelve respuestas
DTO	Define datos de entrada y salida
Service	Aplica reglas de negocio
Repository	Accede a la base de datos
Entity	Representa tablas
Security	Protege rutas y valida JWT
Config	Configuración general y datos iniciales

Ejemplo completo:

POST /api/users

- > UserController.create
- > UserService.createUser
- > PasswordEncoder genera hash
- > UserRepository.save
- > PostgreSQL inserta registro
- > UserResponse vuelve al frontend

20. Funcionamiento de Docker en el proyecto

Docker se usa para ejecutar PostgreSQL de forma simple.

No es la aplicación completa. En desarrollo, Docker solo levanta la base de datos.

Ventajas:

- no instalar PostgreSQL manualmente;
- usar la misma versión en todos los equipos;
- conservar datos con volumen;
- iniciar y detener la BD con comandos simples.

Comandos utiles:

`docker compose up -d`

`docker compose ps`

`docker compose stop`

`docker compose down`

Importante:

`docker compose down -v`

borra también el volumen y por tanto elimina los datos guardados.

21. Funcionamiento de Electron

Electron permite ejecutar la aplicación como programa de escritorio.

Su función principal es abrir una ventana nativa y cargar el frontend.

En desarrollo, normalmente Electron apunta al frontend levantado por Vite.

Por eso, para usar la app desktop, también deben estar levantados:

- PostgreSQL.
- Backend.
- Frontend.
- Electron.

Electron no calcula horarios, no guarda usuarios y no reemplaza la API. Solo envuelve la interfaz.

22. Qué pasa cuando ocurre un error

Los errores pueden ocurrir en distintas capas.

22.1. Error de login

Posibles causas:

- usuario incorrecto;
- contraseña incorrecta;
- usuario desactivado;
- backend apagado;
- base de datos apagada.

22.2. Error 401

Significa que la petición no está autorizada.

Puede pasar si:

- no hay token;
- el token expiro;
- el token es inválido;
- el usuario cerro sesión;
- el backend rechazo la autenticación.

El frontend debe cerrar sesión o redirigir al login.

22.3. Error 403

Significa que el usuario está autenticado, pero no tiene permiso.

Ejemplo:

Usuario USER intenta administrar cuentas

-> backend responde 403

22.4. Error de conexión

Puede pasar si:

- el backend no está en localhost:8081;
- el frontend apunta a una URL incorrecta;
- CORS bloquea la petición;
- PostgreSQL no está disponible.

23. Ejemplo completo: crear un usuario

Este flujo muestra cómo se conectan todas las piezas.

1. Admin abre pantalla Usuarios.
2. Users.jsx pide GET /api/users.
3. api.js agrega Authorization Bearer.
4. UserController valida que sea ADMIN.
5. UserService obtiene lista.
6. UserRepository consulta PostgreSQL.
7. Frontend muestra tabla.
8. Admin llena formulario.
9. Users.jsx envia POST /api/users.
10. Backend valida username único.
11. Backend hashea contraseña con BCrypt.
12. UserRepository guarda registro.
13. Backend devuelve usuario creado.
14. Frontend limpia formulario y refresca tabla.

24. Ejemplo completo: asignar docente a curso

1. Usuario abre Docentes o Cursos.
2. Frontend carga docentes y cursos disponibles.
3. Usuario selecciona docente, curso y turno.
4. Frontend envia la asignación al backend.
5. Backend revisa tipo de docente.
6. Backend revisa categoría del curso.
7. Backend valida ciclo y turno permitido.
8. Backend guarda CourseTeacherAssignment.
9. Frontend refresca docentes y cursos.
10. La futura propuesta de horario ya puede usar esa relación.

25. Ejemplo completo: modificar horarios del día

1. Admin entra a Reglas.
2. Frontend carga GET /api/schedule-settings.
3. Se muestran los seis bloques actuales.
4. Admin cambia una hora.
5. Frontend actualiza visualmente los bloques.
6. Admin presiona Guardar.
7. Backend valida orden, continuidad y duraciones.
8. Backend guarda en schedule_block_settings.
9. La pantalla de Horarios usa esos nuevos rangos.

26. Ejemplo completo: generar horario

1. Usuario entra a Horarios.
2. Frontend solicita la propuesta al backend.
3. Backend obtiene cursos.
4. Backend obtiene docentes asignados.
5. Backend obtiene ambientes.
6. Backend obtiene bloques horarios activos.
7. Backend aplica reglas por ciclo.
8. Backend evita conflictos de docente, ambiente y ciclo.
9. Backend devuelve una propuesta.
10. Frontend la muestra en matriz o vista por franjas.

27. Fuente de verdad del sistema

La fuente de verdad es PostgreSQL.

Esto significa:

- si el dato esta solo en pantalla, todavia no está guardado;
- si el backend no confirma, no debe asumirse que se guardó;
- si se recarga la app, los datos deben venir de PostgreSQL;
- las pantallas deben refrescarse luego de cambios importantes.

El frontend muestra estado temporal, pero la persistencia real está en la base de datos.

28. Resumen final del funcionamiento

Schedule funciona como una aplicación académica por capas.

El usuario opera desde React o Electron. React llama al backend mediante api.js. El backend Spring Boot protege rutas con JWT, aplica reglas de negocio en servicios, usa repositorios JPA para comunicarse con PostgreSQL y devuelve respuestas JSON. PostgreSQL conserva usuarios, docentes, cursos, ambientes, reglas y relaciones.

La parte más importante del sistema es la coordinación entre datos académicos y reglas:

- los cursos tienen ciclos;
- los ciclos determinan turnos;
- los docentes tienen tipos;
- los tipos determinan cursos permitidos;
- los ambientes no deben cruzarse;
- los docentes no deben cruzarse;
- las comidas bloquean franjas no lectivas;
- la propuesta de horario depende de que los datos estén completos.

Por eso Schedule no es solo una interfaz visual. Es un sistema con flujo real de datos, autenticación, persistencia, validaciones y separación clara entre frontend, backend, base de datos y escritorio.